

Module 17: Subqueries

A Subquery is a query nested inside another SQL query. Subqueries allow you to use the result of one query as input to another query, making complex data retrieval and analysis possible.



Introduction

Consider the following question:

Find employees whose salary is greater than the average salary of all employees.

First, we need:

Average Salary

Then:

Compare **every** employee salary

This requires one query inside another query.

This is called a:

Subquery



Learning Objectives

After completing this module, you will be able to:

- ✓ Understand subqueries
- ✓ Write single-row subqueries
- ✓ Write multiple-row subqueries
- ✓ Use subqueries in WHERE
- ✓ Use subqueries in FROM
- ✓ Use subqueries in SELECT

- ✓ Understand correlated subqueries
 - ✓ Use EXISTS and NOT EXISTS
 - ✓ Optimize subquery performance
-



Table of Contents

1. What is a Subquery?
 2. Why Use Subqueries?
 3. Types of Subqueries
 4. Single-Row Subqueries
 5. Multiple-Row Subqueries
 6. Subqueries with IN
 7. Subqueries with EXISTS
 8. Correlated Subqueries
 9. Subqueries in SELECT
 10. Subqueries in FROM
 11. Nested Subqueries
 12. IN vs EXISTS
 13. Common Mistakes
 14. Best Practices
 15. Interview Questions
 16. Summary
 17. Practice Questions
-

1 What is a Subquery?

A subquery is a query inside another query.

General Structure

```
SELECT ColumnName
FROM TableName
WHERE ColumnName =
(
    SELECT Expression
    FROM TableName
);
```

Example

```
SELECT EmployeeName
FROM Employees
WHERE Salary >
(
    SELECT AVG(Salary)
    FROM Employees
);
```

Execution

Step 1:

```
SELECT AVG(Salary)
FROM Employees;
```

Result:

```
50000
```

Step 2:

```
SELECT EmployeeName
FROM Employees
WHERE Salary > 50000;
```

Final Result:

Employees earning above average.

Why Subqueries Matter

Used in:

- Reporting
- Analytics
- Data Validation
- Dashboard Queries
- Interview Problems

2 Why Use Subqueries?

Without subqueries:

```
Run Query 1
Store Result
Run Query 2
```

With subqueries:

```
Single SQL Statement
```

Example

Find highest-paid employee.

Without Subquery

Step 1:

```
SELECT MAX(Salary)
FROM Employees;
```

Step 2:

```
SELECT *
FROM Employees
WHERE Salary = 80000;
```

With Subquery

```
SELECT *
FROM Employees
WHERE Salary =
(
    SELECT MAX(Salary)
```

```
FROM Employees  
);
```

3 Types of Subqueries

Type	Returns
Single-Row Subquery	One value
Multiple-Row Subquery	Multiple values
Correlated Subquery	Depends on outer query
Nested Subquery	Subquery inside subquery

4 Single-Row Subqueries

Returns exactly one value.

Example

Find employees earning above average salary.

```
SELECT *  
FROM Employees  
WHERE Salary >  
(  
    SELECT AVG(Salary)  
    FROM Employees  
);
```

Inner Query:

```
AVG(Salary)
```

returns:

```
50000
```

Outer Query:

```
Salary > 50000
```

Example

Find youngest employee.

```
SELECT *  
FROM Employees  
WHERE Age =  
(  
    SELECT MIN(Age)  
    FROM Employees  
);
```

Common Operators

```
=  
>  
<  
>=  
<=  
<>
```

5 Multiple-Row Subqueries

Returns multiple values.

Example

Departments

DepartmentID

1

2

Query:

```
SELECT DepartmentID
FROM Departments
WHERE Location = 'Delhi';
```

Returns:

```
1
2
```

Outer Query:

```
SELECT *
FROM Employees
WHERE DepartmentID IN
(
    SELECT DepartmentID
    FROM Departments
    WHERE Location='Delhi'
);
```

Why IN?

Because multiple values are returned.

Bad:

```
=
(
    SELECT DepartmentID
    ...
)
```

Error if multiple rows returned.

Subqueries with IN

Most common multiple-row subquery.

Example

```
SELECT *
FROM Employees
WHERE DepartmentID IN
(
    SELECT DepartmentID
    FROM Departments
    WHERE Location='Delhi'
);
```

NOT IN

```
SELECT *
FROM Employees
WHERE DepartmentID NOT IN
(
    SELECT DepartmentID
    FROM Departments
    WHERE Location='Delhi'
);
```

Uses

- Category filtering
 - Product filtering
 - Customer filtering
-

Subqueries with EXISTS

Checks whether rows exist.

Syntax

```
WHERE EXISTS
(
```



```
Subquery  
)
```

Example

Find customers with orders.

```
SELECT *  
FROM Customers C  
WHERE EXISTS  
(  
    SELECT *  
    FROM Orders O  
    WHERE O.CustomerID = C.CustomerID  
);
```

Logic

If matching order exists:

```
TRUE
```

Customer returned.

Otherwise:

```
FALSE
```

Customer excluded.

NOT EXISTS

Find customers without orders.

```
SELECT *  
FROM Customers C  
WHERE NOT EXISTS  
(  
    SELECT *
```

```
FROM Orders O
WHERE O.CustomerID = C.CustomerID
);
```

8 Correlated Subqueries

Advanced topic.

A correlated subquery depends on the outer query.

Example

Find employees earning above department average.

```
SELECT *
FROM Employees E
WHERE Salary >
(
    SELECT AVG(Salary)
    FROM Employees
    WHERE DepartmentID = E.DepartmentID
);
```

How It Works

For each employee:

SQL calculates:

```
Department Average
```

Then compares.

Example

IT Average:

```
50000
```

Employee Salary:

60000

Result:

Included.

Characteristics

✓ References outer query

✓ Executes repeatedly

✗ Can be slower

Subqueries in SELECT

Subqueries can appear in SELECT.

Example

```
SELECT EmployeeName,  
  
  (  
    SELECT AVG(Salary)  
    FROM Employees  
  ) AS CompanyAverage  
  
FROM Employees;
```

Output

EmployeeName	CompanyAverage
Kunal	50000
Rahul	50000

Uses

- KPI Reports
 - Dashboard Metrics
-

10 Subqueries in FROM

A subquery can act like a temporary table.

Example

```
SELECT *
FROM
(
    SELECT DepartmentID,
           AVG(Salary) AS AvgSalary
    FROM Employees
    GROUP BY DepartmentID
) AS DeptAvg;
```

Result

DepartmentID	AvgSalary
1	50000
2	60000

Called

Derived Table

1 1 Nested Subqueries

Subquery inside another subquery.

Example

```
SELECT *  
FROM Employees  
WHERE DepartmentID IN  
(  
    SELECT DepartmentID  
    FROM Departments  
    WHERE LocationID IN  
    (  
        SELECT LocationID  
        FROM Locations  
        WHERE Country='India'  
    )  
);
```

Execution:

```
Inner Query  
→ Middle Query  
→ Outer Query
```

1 2 IN vs EXISTS

Important interview topic.

IN

```
WHERE DepartmentID IN (...)
```

Compares values.

EXISTS

```
WHERE EXISTS (...)
```

Checks existence.

Comparison

Feature	IN	EXISTS
Compares Values	Yes	No
Checks Existence	No	Yes
Good for Small Results	Yes	Yes
Better for Large Data	Sometimes	Often

Example

IN:

```
WHERE DepartmentID IN
(
  SELECT DepartmentID
  FROM Departments
)
```

EXISTS:

```
WHERE EXISTS
(
  SELECT *
  FROM Departments
)
```

1 **3** Common Mistakes

Using = with Multiple Rows

Bad:

```
WHERE DepartmentID =
(
  SELECT DepartmentID
```

```
)  
FROM Departments
```

Error:

```
Subquery returned multiple rows.
```

Use:

```
IN
```

Missing Alias in FROM Subquery

Bad:

```
FROM  
(  
  SELECT ...  
)
```

Correct:

```
FROM  
(  
  SELECT ...  
)  
AS TempTable
```

Correlated Query Performance

May become slow on large datasets.

Using NOT IN with NULL

Can produce unexpected results.

Prefer:

NOT EXISTS

1 4 Best Practices

Use EXISTS for Large Data

Efficient for existence checks.

Use IN for Small Lists

Easy to read.

Avoid Deep Nesting

Too many nested queries reduce readability.

Consider JOINS

Sometimes JOINS outperform subqueries.

Format Properly

Readable SQL is maintainable SQL.

Professional Example

```
SELECT EmployeeID,  
       EmployeeName,  
       Salary  
FROM Employees  
WHERE Salary >  
(  
    SELECT AVG(Salary)  
    FROM Employees  
);
```

1 5 Common Interview Questions

What is a Subquery?

A query inside another query.

Difference Between IN and EXISTS?

IN compares values.

EXISTS checks existence.

What is a Correlated Subquery?

Depends on the outer query.

Can Subqueries Be Used in SELECT?

Yes.

Can Subqueries Be Used in FROM?

Yes.

Which is Faster?

Depends on:

- Data size
 - Indexes
 - Query plan
-

Business Analytics Examples

Employees Above Average Salary

```
SELECT *  
FROM Employees
```

```
WHERE Salary >
(
  SELECT AVG(Salary)
  FROM Employees
);
```

Customers with Orders

```
SELECT *
FROM Customers C
WHERE EXISTS
(
  SELECT *
  FROM Orders O
  WHERE O.CustomerID = C.CustomerID
);
```

Products Never Sold

```
SELECT *
FROM Products P
WHERE NOT EXISTS
(
  SELECT *
  FROM Sales S
  WHERE S.ProductID = P.ProductID
);
```

Departments Above Average Revenue

```
SELECT DepartmentID,  
       SUM(Sales)  
FROM Sales  
GROUP BY DepartmentID  
HAVING SUM(Sales) >  
(  
    SELECT AVG(TotalSales)  
    FROM DepartmentSales  
);
```



Summary

In this module, you learned:

- ✓ Single-Row Subqueries
- ✓ Multiple-Row Subqueries
- ✓ IN
- ✓ EXISTS
- ✓ NOT EXISTS
- ✓ Correlated Subqueries
- ✓ Subqueries in SELECT
- ✓ Subqueries in FROM
- ✓ Nested Subqueries
- ✓ IN vs EXISTS



Practice Questions

Theory

1. What is a subquery?
2. Why are subqueries used?
3. What is a single-row subquery?
4. What is a multiple-row subquery?
5. Difference between IN and EXISTS?
6. What is a correlated subquery?
7. Can subqueries be used in SELECT?
8. Can subqueries be used in FROM?

9. What is a derived table?
 10. Why can correlated subqueries be slow?
-

Practical Exercises

Task 1

Find employees earning above average salary.

Task 2

Find highest-paid employee using a subquery.

Task 3

Find customers with orders using EXISTS.

Task 4

Find customers without orders using NOT EXISTS.

Task 5

Create a derived table using a FROM subquery.

Task 6

Write a correlated subquery.

Challenge Project

Create:

```
Customers
Orders
Products
Sales
Departments
```

Generate reports:

- Customers with Orders
- Products Never Sold
- Employees Above Department Average

- Departments Above Average Revenue

using:

- IN
- EXISTS
- NOT EXISTS
- Correlated Subqueries
- Derived Tables



Next Module

→ Module 18: CASE Statement

Topics Covered:

- Simple CASE
- Searched CASE
- CASE in SELECT
- CASE in WHERE
- CASE in ORDER BY
- Conditional Aggregation
- Business Reporting Examples
- Interview Questions